

AutoCourseReminder

Documentation technique

Plugin ILIAS 10.5 - Version 1.1.0

Identifiant plugin	acrm
Type / slot ILIAS	User Interface Hook Plugin avec fournisseur de tâche cron
Version cible	ILIAS 10.5 ; base de données testée : MariaDB
Dépôt	github.com/vincent-sayah/AutoCourseReminder

Objet du document

Cette documentation décrit l'architecture, le modèle de données, les flux d'exécution, les points d'intégration ILIAS, les mécanismes de sécurité et les limites fonctionnelles du plugin AutoCourseReminder tel qu'il est implémenté dans le dépôt GitHub final v1.1.0.

1. Synthèse d'architecture

AutoCourseReminder est un plugin ILIAS de type User Interface Hook. Il combine trois responsabilités principales : le suivi de l'activité utilisateur pendant la navigation dans les cours, l'administration des paramètres de rappel, et l'exécution d'un job cron qui évalue les règles puis crée des notifications dans la messagerie interne ILIAS.

Composant	Rôle technique	Points clés
plugin.php	Manifeste du plugin	Déclare l'identifiant acrm, la version 1.1.0 et la compatibilité ILIAS 10.x.
ilAutoCourseReminderPlugin	Point d'entrée principal	Initialise les classes, expose le repository, le service métier et le fournisseur de cron.
ilAutoCourseReminderUIHookGUI	Entrée côté interface	Traite une fois par requête la désinscription et le suivi d'activité, puis délègue l'UI des paramètres de cours.
ilAutoCourseReminderCourseSettingsGUI	UI des règles de cours	Ajoute un sous-onglet dans Paramètres, rend le formulaire et remplace le contenu central de la page.
ilAutoCourseReminderConfigGUI	UI de configuration globale	Gère base_url, client_id, mail_from et mail_from_name.
ilAutoCourseReminderService	Service métier	Évalue l'éligibilité, construit les messages, génère les liens de désinscription et déclenche l'envoi.
ilAutoCourseReminderRepository	Accès aux données	Lit et écrit les réglages, l'activité, l'historique d'envoi, les désinscriptions et les règles par cours.
ilAutoCourseReminderCronJob	Tâche planifiée	Expose l'ID de job, les types de planification autorisés et l'exécution manuelle.
ilAutoCourseReminderMailer / InternalMailNotification	Adaptation vers la messagerie ILIAS	Créent une notification interne ILIAS et ajoutent la signature d'installation.

Note technique - Le service et le repository conservent également une compatibilité arrière avec un ancien mode de configuration JSON (clé `rules_json`) et avec un type de règle "step", alors que l'interface courante de la version 1.1.0 expose uniquement une règle d'inactivité par cours.

2. Arborescence du dépôt

Répertoire / fichier	Contenu
classes/	Implémentation PHP des interfaces UI, du service métier, du repository, du cron et de l'adaptateur de messagerie.
lang/	Fichiers de langue français et anglais pour l'interface d'administration.
sql/dbupdate.php	Définition et création incrémentale des tables utilisées par le plugin.
plugin.php	Métadonnées de version et de compatibilité.
README.md	Description fonctionnelle, installation, configuration et exemples de test SQL.

3. Intégration ILIAS

- Le plugin hérite de `ilUserInterfaceHookPlugin` et s'insère donc dans le slot User Interface Hook.
- Il implémente également `ilCronJobProvider` afin d'exposer une tâche planifiée exécutable manuellement ou selon une planification souple.
- La logique métier exploite plusieurs composants ILIAS natifs : `ilObject`, `ilLPStatusWrapper`, `ilCourseParticipants`, `ilObjUser`, `ilMailNotification`, `ilUtil` et les services DIC (logger, access, language, repositoryTree, user).
- Le paramétrage global s'effectue dans l'administration du plugin ; le paramétrage métier des rappels s'effectue au niveau de chaque cours.

Le choix du slot UI Hook permet au plugin d'intercepter des requêtes sans modifier le cœur d'ILIAS. Cette approche limite le couplage mais implique un travail DOM côté HTML pour injecter le sous-écran de configuration de cours.

4. Configuration globale du plugin

La classe `ilAutoCourseReminderConfigGUI` construit un formulaire d'administration avec deux blocs : paramètres techniques et rappel que la configuration fonctionnelle se fait désormais dans chaque cours.

Clé	Usage
-----	-------

base_url	URL de base utilisée pour construire les URL goto et les liens de désinscription ; si vide, le service tente d'utiliser ILIAS_HTTP_PATH.
client_id	Ajouté à l'URL goto si la constante CLIENT_ID n'est pas disponible.
mail_from	Adresse expéditeur conservée dans les réglages et renvoyée par le service lors de la préparation du message.
mail_from_name	Nom expéditeur injecté dans la variable de template {{MAIL_FROM_NAME}}.
token_secret	Secret HMAC généré automatiquement au premier enregistrement ou lors du premier passage du cron s'il n'existe pas.
rules_json	Ancien format de configuration ; toujours lu en fallback si aucune table ui_uihk_acrm_crule n'est utilisée.

Observation d'implémentation - Dans l'adaptateur de messagerie actuel, from_address et from_name ne pilotent pas l'expéditeur interne ILIAS. Ils sont surtout utiles pour le rendu des templates et pour une éventuelle évolution future vers un autre canal d'envoi.

5. Configuration par cours

La classe ilAutoCourseReminderCourseSettingsGUI ajoute un sous-onglet "Rappels automatiques" dans la zone Paramètres du cours, à condition que l'utilisateur courant dispose du droit d'écriture sur le cours.

- active : active ou non les rappels pour le cours.
- delay_days : nombre de jours avant le premier rappel.
- repeat_every_days : délai minimal entre deux rappels.
- max_reminders : nombre maximal de rappels par utilisateur et par cours.
- allow_opt_out : ajoute un lien de désinscription dans le message.
- subject / body : gabarits de message paramétrables avec variables de personnalisation.

Les champs numériques sont validés comme entiers strictement positifs. En cas d'erreur, l'écran reste sur le formulaire et affiche les alertes de validation.

L'interface remplace le contenu du centre de page via DOMDocument, puis vide explicitement les colonnes gauche et droite afin d'obtenir un écran de configuration focalisé.

Particularité UI - La classe encapsule les marqueurs {{VARIABLE}} en jetons temporaires puis les ré-encode en HTML ({{ et }}) afin d'éviter leur altération par le

rendu du formulaire. Ce mécanisme a été introduit pour préserver l’affichage correct des doubles accolades dans l’UI.

6. Variables de template

Variable	Valeur injectée
{{FIRSTNAME}}	Prénom du destinataire.
{{LASTNAME}}	Nom du destinataire.
{{LOGIN}}	Login ILIAS du destinataire.
{{COURSE_TITLE}}	Titre du cours.
{{COURSE_URL}}	URL goto vers le cours, avec client_id si nécessaire.
{{INACTIVITY_DAYS}}	Nombre de jours calculé depuis last_seen.
{{MAIL_FROM_NAME}}	Nom expéditeur défini dans les réglages globaux.
{{OPTOUT_BLOCK}}	Bloc de texte contenant le lien de désinscription si allow_opt_out est activé ; chaîne vide sinon.
{{DISABLE_URL}}	URL brute de désinscription, disponible côté service même si l’aide UI n’affiche pas explicitement cette variable.

7. Modèle de données

Le fichier sql/dbupdate.php crée cinq tables métier. Toutes les écritures applicatives passent ensuite par le repository en utilisant majoritairement la méthode ILIAS replace, ce qui fournit un comportement d’upsert.

Table	Rôle	Clé primaire / index
ui_uihk_acrm_settings	Réglages globaux du plugin.	PK setting_key
ui_uihk_acrm_activity	Première et dernière activité d’un utilisateur dans un cours.	PK (user_id, course_ref_id) ; index (course_ref_id, last_seen)
ui_uihk_acrm_dispatch	Historique d’envoi par utilisateur et par règle.	PK (user_id, rule_key)
ui_uihk_acrm_optout	Désinscription d’un	PK (user_id, course_ref_id)

	utilisateur pour un cours.	
ui_uihk_acrm_crule	Règle d'inactivité paramétrée pour un cours.	PK course_ref_id

7.1 Détail des tables

Table	Colonnes principales	Commentaires
ui_uihk_acrm_settings	setting_key, setting_value	Table clé/valeur. setting_value est de type CLOB.
ui_uihk_acrm_activity	user_id, course_ref_id, first_seen, last_seen	first_seen reste figé au premier passage ; last_seen est mis à jour à chaque activité détectée.
ui_uihk_acrm_dispatch	user_id, rule_key, sent_count, last_sent, last_reason	Permet le plafond de relance et le délai minimal entre envois.
ui_uihk_acrm_optout	user_id, course_ref_id, created_at	Empêche tout nouvel envoi sur le cours concerné.
ui_uihk_acrm_crule	course_ref_id, active, delay_days, repeat_every_days, max_reminders, allow_opt_out, subject, body	La version UI actuelle ne stocke qu'une règle d'inactivité par cours.

8. Flux d'exécution

8.1 Suivi d'activité pendant la navigation

1. Une requête front ILIAS arrive sur le hook UI.
2. ilAutoCourseReminderUIHookGUI exécute une seule fois handleOptOutRequest() puis trackCurrentRequest() pour la requête.
3. trackCurrentRequest() lit ref_id, ignore les utilisateurs anonymes et résout le cours parent si l'objet courant n'est pas lui-même un cours.
4. Le repository enregistre ou met à jour ui_uihk_acrm_activity avec first_seen si absent et last_seen à l'horodatage courant.

Conséquence importante : un utilisateur ne peut être rappelé qu'après avoir généré au moins une activité détectable dans le cours ou dans un objet situé sous ce cours.

8.2 Désinscription

5. Le message peut contenir un lien vers `ilias.php?baseClass=ilDashboardGUI` avec les paramètres `acr_action=disable`, `acr_u`, `acr_c` et `acr_t`.
6. `acr_t` est un HMAC SHA-256 calculé sur la chaîne "user_id:course_ref_id" avec le secret `token_secret`.
7. Si le jeton est valide, le repository enregistre la ligne de désinscription dans `ui_uihk_acrm_optout` puis redirige vers le cours.
8. Si le jeton est invalide, le plugin journalise un warning et redirige également vers le cours.

Point de sécurité - Le jeton ne porte ni date d'expiration ni nonce. Il reste donc valide tant que `token_secret` ne change pas et que le couple utilisateur/cours reste identique.

8.3 Évaluation par le cron

9. Le job cron `autocoursereminder_job` appelle le service métier.
10. Le service charge les règles actives via `getRules()`.
11. Pour chaque cours, il récupère les membres, les utilisateurs en progression, les utilisateurs déjà complétés et, en mode legacy step, les complétions d'étape.
12. Pour chaque membre, il applique les filtres : progression en cours, cours non complété, non désinscrit, activité connue, règle applicable, plafond de rappels non atteint, délai de relance respecté.
13. Le service construit ensuite le sujet, le corps et le lien goto, puis demande à l'adaptateur de créer une notification interne.
14. En cas de succès, le repository met à jour `ui_uihk_acrm_dispatch`.

9. Algorithme d'éligibilité

Condition	Règle d'inactivité	Règle de type step (compatibilité legacy)
Membre du cours	Oui	Oui
Statut LP requis	In progress	In progress
Cours déjà complété	Exclusion	Exclusion
Désinscription	Exclusion	Exclusion
Présence d'activité	<code>first_seen</code> et <code>last_seen</code> requis	<code>first_seen</code> et <code>last_seen</code> requis
Seuil temporel	<code>now - last_seen >= delay_days</code>	<code>now - first_seen >= delay_days</code>
Complétion de l'étape	Non applicable	Exclusion si l'étape ciblée est déjà complétée
Rythme de relance	<code>now - last_sent >= repeat_every_days</code>	Même logique

Plafond	sent_count < max_reminders	Même logique
---------	----------------------------	--------------

10. Messagerie et notifications

L'envoi effectif passe par `ilAutoCourseReminderMailAdapter`, qui instancie `ilAutoCourseReminderInternalMailNotification`. Cette dernière hérite de `ilMailNotification`, charge la langue du destinataire, initialise le mail interne, affecte le sujet et le corps, ajoute deux sauts de ligne puis la signature d'installation, et envoie le message au destinataire.

- Canal actuel : messagerie interne ILIAS.
- Journalisation : info en cas de succès, warning si le login du destinataire est vide, error sur exception.
- Le sujet et le corps sont générés côté service avant l'appel à l'adaptateur.

11. Valeurs par défaut et mécanismes de normalisation

Paramètre	Valeur par défaut
active	false
delay_days	5
repeat_every_days	5
max_reminders	3
allow_opt_out	true
subject	[ILIAS] Rappel d'activité - {{COURSE_TITLE}}
body	Message d'inactivité en français avec {{FIRSTNAME}}, {{COURSE_TITLE}}, {{INACTIVITY_DAYS}}, {{COURSE_URL}}, {{OPTOUT_BLOCK}} et {{MAIL_FROM_NAME}}

Le repository détecte également d'anciens modèles défectueux contenant des placeholders "{}" et les remplace automatiquement par les modèles corrigés fondés sur les doubles accolades.

Compatibilité ascendante - Cette normalisation protège la migration depuis des versions où le sujet ou le corps par défaut n'étaient pas correctement formatés.

12. Sécurité, contrôle d'accès et robustesse

- Accès à l'interface de réglage de cours limité au droit write sur le cours.
- Exclusion des utilisateurs anonymes du suivi d'activité.
- Contrôle d'intégrité du lien de désinscription par HMAC SHA-256.

- Aucune suppression logique des données d'activité ou d'historique n'est automatisée.
- Aucune expiration native des tokens de désinscription.
- Aucun canal SMTP externe spécifique n'est géré par cette version ; le plugin s'appuie sur les mécanismes internes ILIAS.

13. Journalisation et observabilité

Le plugin écrit des messages dans le logger racine ILIAS. Les événements principaux observables sont les suivants :

- Démarrage du job cron.
- Cours introuvable pour une règle.
- Nombre de vérifications et nombre de notifications envoyées en fin de job.
- Succès de création d'une notification interne pour un utilisateur donné.
- Échec d'envoi sur exception.
- Lien de désactivation invalide.
- Désactivation enregistrée.

La version 1.1.0 ne propose pas encore d'écran d'administration dédié pour consulter l'historique détaillé ; l'analyse passe donc par les logs et par les tables SQL.

14. Installation et exploitation

Le README du projet prévoit un déploiement dans le répertoire standard des plugins UI Hook puis une installation/activation via l'administration ILIAS. Il mentionne également une mise à jour des artefacts ILIAS depuis la racine de l'installation.

```
public/Customizing/global/plugins/Services/UIComponent/UserInterfaceHook/
AutoCourseReminder
```

Après activation, deux plans de configuration doivent être traités :

- Configuration technique globale dans l'administration du plugin.
- Configuration métier dans chaque cours concerné, sous Paramètres > Rappels automatiques.

Le job cron doit ensuite être planifié dans ILIAS. Les types autorisés par la classe cron sont : journalier, tous les N jours, ou toutes les N heures. La valeur par défaut est une exécution quotidienne.

15. Tests techniques recommandés

Le README fournit des exemples SQL utiles pour valider l'implémentation en environnement d'intégration ou de recette.

```
UPDATE ui_uihk_acrm_activity
SET last_seen = NOW() - INTERVAL 2 DAY
WHERE user_id = 328
  AND course_ref_id = 89;

DELETE FROM ui_uihk_acrm_dispatch
WHERE user_id = 328
```

```

AND rule_key = 'course_89_inactivity';

SELECT *
FROM ui_uihk_acrm_optout
WHERE user_id = 328
      AND course_ref_id = 89;

```

- Tester un utilisateur en progression avec activité déjà tracée.
- Vérifier qu'un utilisateur complétant le cours ne reçoit plus de relance.
- Vérifier qu'un utilisateur désinscrit ne reçoit plus de relance.
- Vérifier le comportement après rotation de token_secret si vous souhaitez invalider tous les liens existants.

16. Limites fonctionnelles et techniques

- Une seule règle de type inactivité est exposée par cours dans l'interface de la version 1.1.0.
- Il n'existe pas encore de journal d'administration détaillé des envois.
- La désinscription est unidirectionnelle : le dépôt n'expose pas de réactivation UI native.
- Les tokens de désinscription n'expirent pas d'eux-mêmes.
- L'insertion UI repose sur la structure HTML des pages de paramètres de cours ; toute évolution majeure du markup ILIAS peut nécessiter un ajustement du hook DOM.
- Le support des règles "step" subsiste dans le service et le repository mais n'est pas pilotable depuis l'interface de cours actuelle.

17. Pistes d'évolution

- Ajouter une interface d'administration centralisée de suivi des envois et des désinscriptions.
- Introduire plusieurs règles par cours, y compris des règles basées sur des objets ou étapes cibles.
- Ajouter l'expiration ou la révocation ciblée des liens de désinscription.
- Prévoir une purge des tables activity et dispatch selon une politique de rétention.
- Rendre le canal de notification abstrait afin de pouvoir cibler d'autres moyens d'envoi à terme.

18. Annexe - Cartographie fichier par fichier

Fichier	Responsabilité principale	Méthodes ou éléments notables
plugin.php	Métadonnées du plugin	id=acrm ; version 1.1.0 ; plage ILIAS 10.0-10.999.
class.ilAutoCourseReminderPlugin.php	Bootstrap et factory	init(), getCronJobInstances(), getRepository(), getService().
class.ilAutoCourseReminderUIHookGUI.php	Hook UI	getHTML(), modifyGUI().

class.ilAutoCourseReminderConfigGUI.php	Configuration globale	performCommand(), buildForm(), save().
class.ilAutoCourseReminderCourseSettingsGUI.php	Écran de paramètres de cours	Injection DOM, validation, aide template, URL <code>acr_course_settings=1</code> .
class.ilAutoCourseReminderRepository.php	Persistence	getCourseRule(), saveCourseRule(), touchCourseActivity(), saveDispatch(), saveOptOut().
class.ilAutoCourseReminderService.php	Service métier	run(), trackCurrentRequest(), handleOptOutRequest(), buildMessage().
class.ilAutoCourseReminderCronJob.php	Définition du cron	autocoursereminder_job ; planification flexible.
class.ilAutoCourseReminderMailAdapter.php	Adaptation d'envoi	sendToUser().
class.ilAutoCourseReminderInternalMailNotification.php	Notification interne	sendReminder() via ilMailNotification.
sql/dbupdate.php	Schéma de base	Création des 5 tables.
lang/ilias_fr.lang & ilias_en.lang	Libellés UI	Textes de configuration, aide et validation.

19. Conclusion

La version 1.1.0 d'AutoCourseReminder présente une architecture simple et cohérente pour ILIAS 10.5 : persistance dédiée, configuration à deux niveaux, suivi d'activité embarqué dans le hook UI, et envoi par la messagerie interne. Le code contient déjà plusieurs indices de maturité technique - compatibilité legacy, normalisation des templates, sécurisation HMAC, upserts via le repository - tout en laissant visibles des axes d'évolution pour une future version plus riche.